

Scalable Technologies for Data Analysis

Introduction / Presentation

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026

Scalable Technologies for Data Analysis

"*tekhne*" (technique, arts, craft) + "*logia*" (study of)
A product of science and engineering that involves a set of instruments, methods and techniques aimed at solving problems.

Scalable Technologies for Data Analysis

1. That can be climbed (e.g., climbable wall).
2. That is able to grow uniformly or to withstand an increase in load.

"*tekhne*" (technique, arts, craft) + "*logia*" (study of)
A product of science and engineering that involves a set of instruments, methods and techniques aimed at solving problems.

Scalable Technologies for Data Analysis

1. That can be climbed (e.g., climbable wall).
2. That is able to grow uniformly or to withstand an increase in load.

"*tekhne*" (technique, arts, craft) + "*logia*" (study of)
A product of science and engineering that involves a set of instruments, methods and techniques aimed at solving problems.

Scalable Technologies for Data Analysis

A process through which data is inspected, cleaned, transformed, and modeled in order to discover useful information, suggest conclusions, and support decision-making processes

1. The big (data) picture

Literally...

— INDUSTRY

MACHINE LEARNING & ARTIFICIAL INTELLIGENCE

BUILD

AI OBSERVABILITY & EVALUATION

AI DEVELOPER PLATFORMS

LOCAL/ON-DEVICE LLM RUNTIMES

AGENT PLATFORMS

AGENT INFRA / TOOLING

VALIDATE & SECURE

AI OBSERVABILITY & EVALUATION

AI SAFETY & SECURITY

ROUTING, PROMPT MGT & EXPERIMENTATION

AI & AGENT GOVERNANCE

OPERATE & SUPPLY

ML OPS

DATA GENERATION & LABELING

DATA SCIENCE

ENTERPRISE AI PLATFORMS

MODALITIES & RESEARCH

SPEECH & VOICE AI

COMPUTER VISION

AI R&D — COMMERCIAL

AI R&D — NONPROFIT / ACADEMIC

RUN & DEPLOY

GPU CLOUD / DEPLOYMENT INFRA

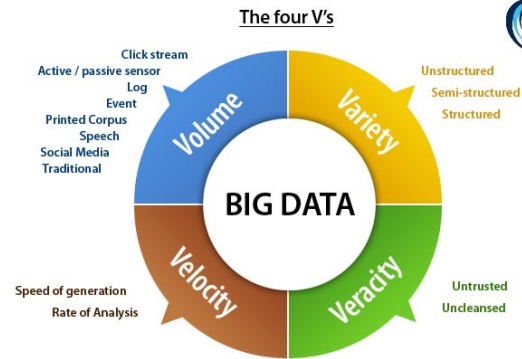
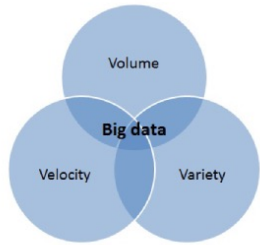
MODEL SERVING & INFERENCE RUNTIMES

AI HARDWARE

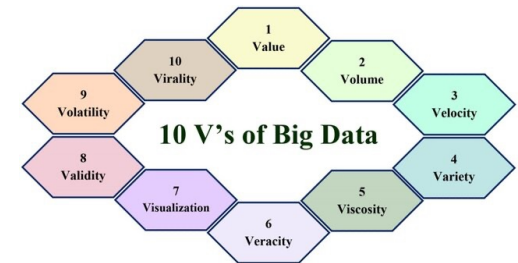
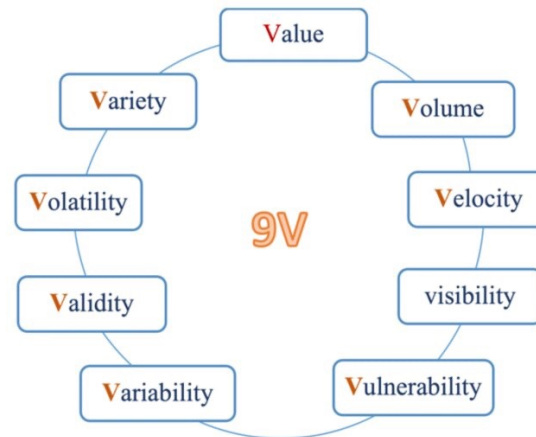
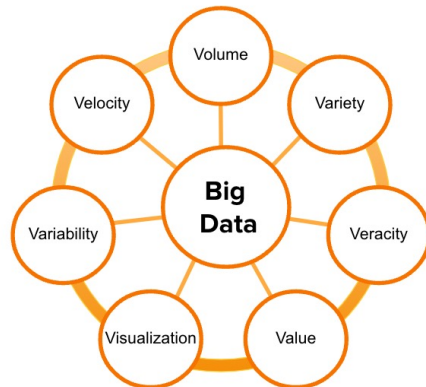
EDGE AI

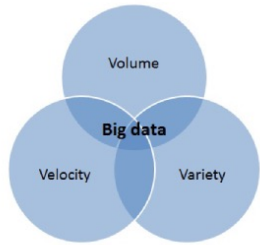
— DATA & AI CONSULTING

FIRSTMARK

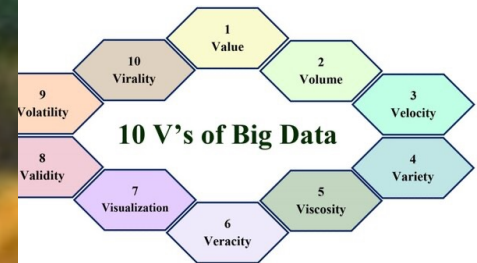
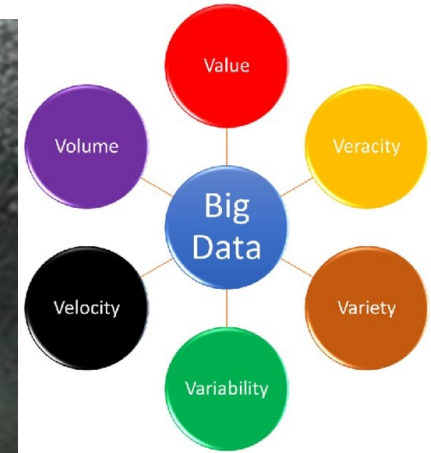
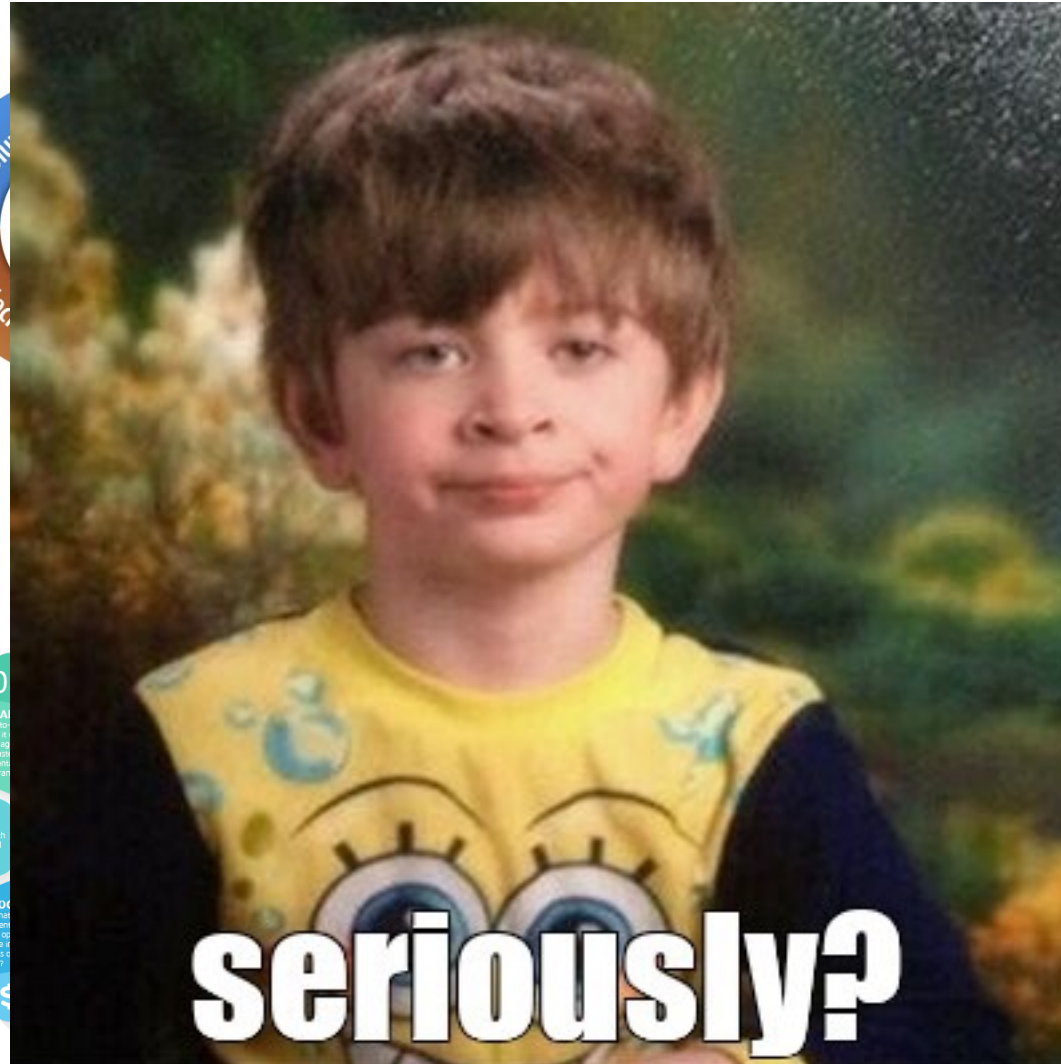
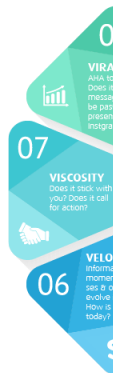
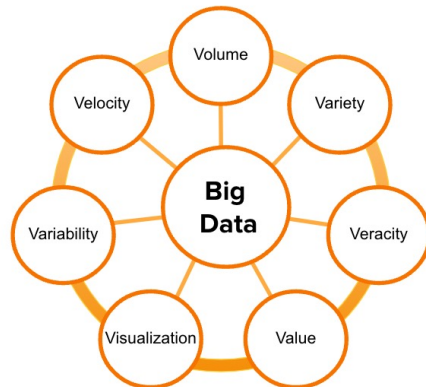


7 V'S OF BIG DATA





7 V'S OF BIG DATA

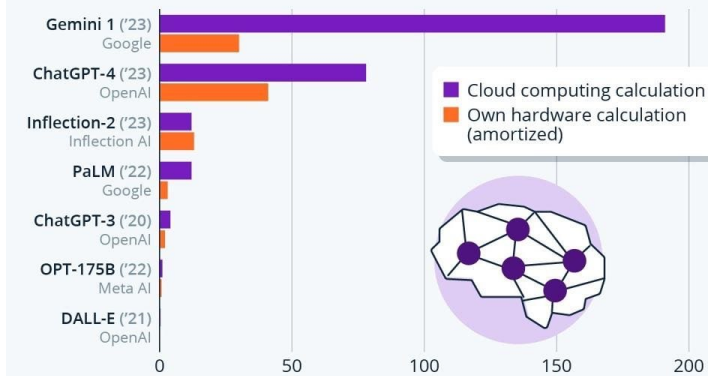


The core problem

- Scalability is not just “more data”
 - **Volume:** data grows faster than teams
 - **Velocity:** updates are continuous, not monthly
 - **Variety:** logs, events, tables, files, APIs
- There are now real-world constraints and requirements
 - Cost
 - Reliability
 - Governance
 - Security
 - ...

The Extreme Cost Of Training AI Models

Estimated cost of training selected AI models (in million U.S. dollars), by different calculation models



Rounded numbers. Excludes staff salaries that can make up 29-49% of final cost (including equity)
Source: Epoch AI



statista

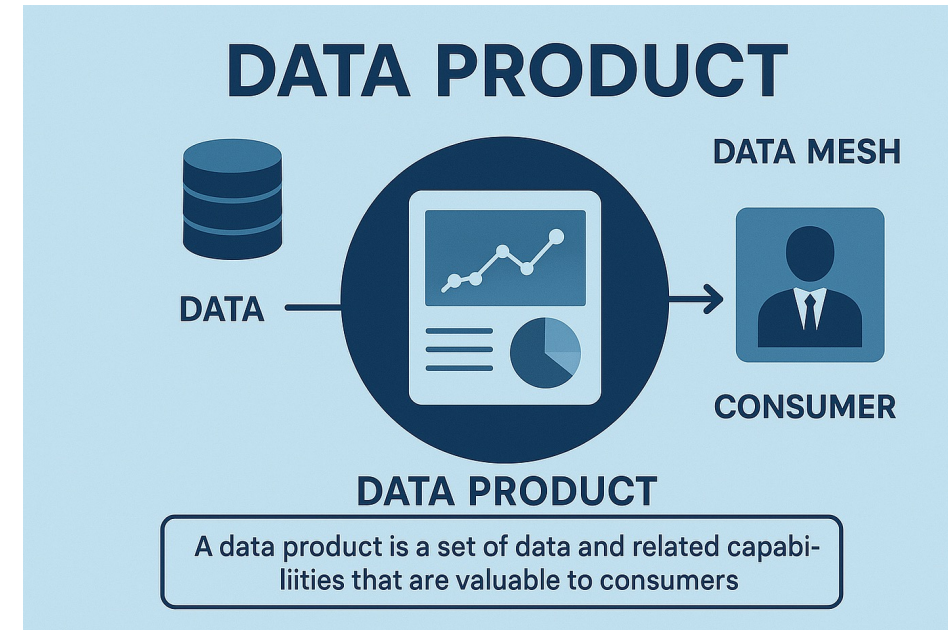


What does scalable mean? Why companies care?

- A scalable solution
 - Keeps working as data + users grow
 - Has predictable cost/performance trade-offs
 - Is observable + debuggable in production
 - Makes changes safely (schemas, pipelines, metrics)
- Scalability is technical + organizational
- Scalable data platforms enable
 - Faster decision cycles from operations to strategy
 - Consistent KPIs, i.e. “one version of truth”
 - Reliable data products (e.g. recommendations, risk, forecasting)
 - Regulatory + audit requirements (who accessed what, when)

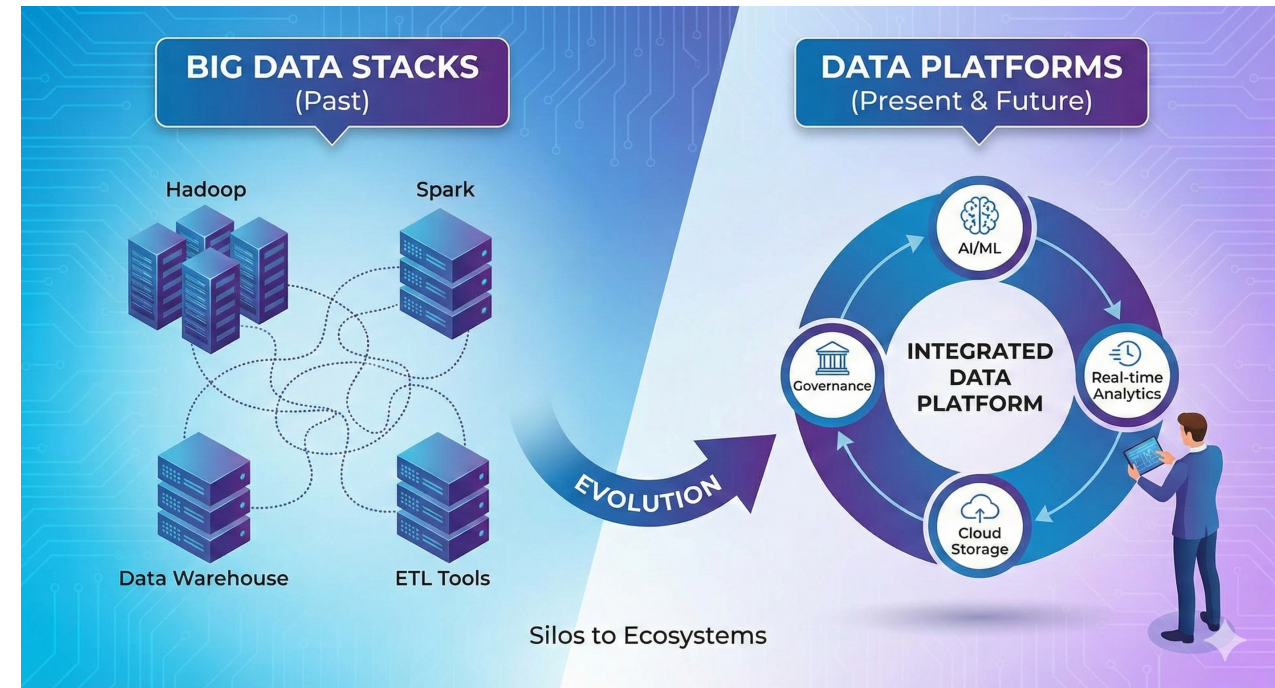
From outputs to **data products**

- We'll treat data as a product, not as a generic dataset
 - Defined consumers + use cases
 - Clear semantics (metrics, dimensions)
 - Contracts and SLAs/SLOs
 - Versioning, documentation, ownership



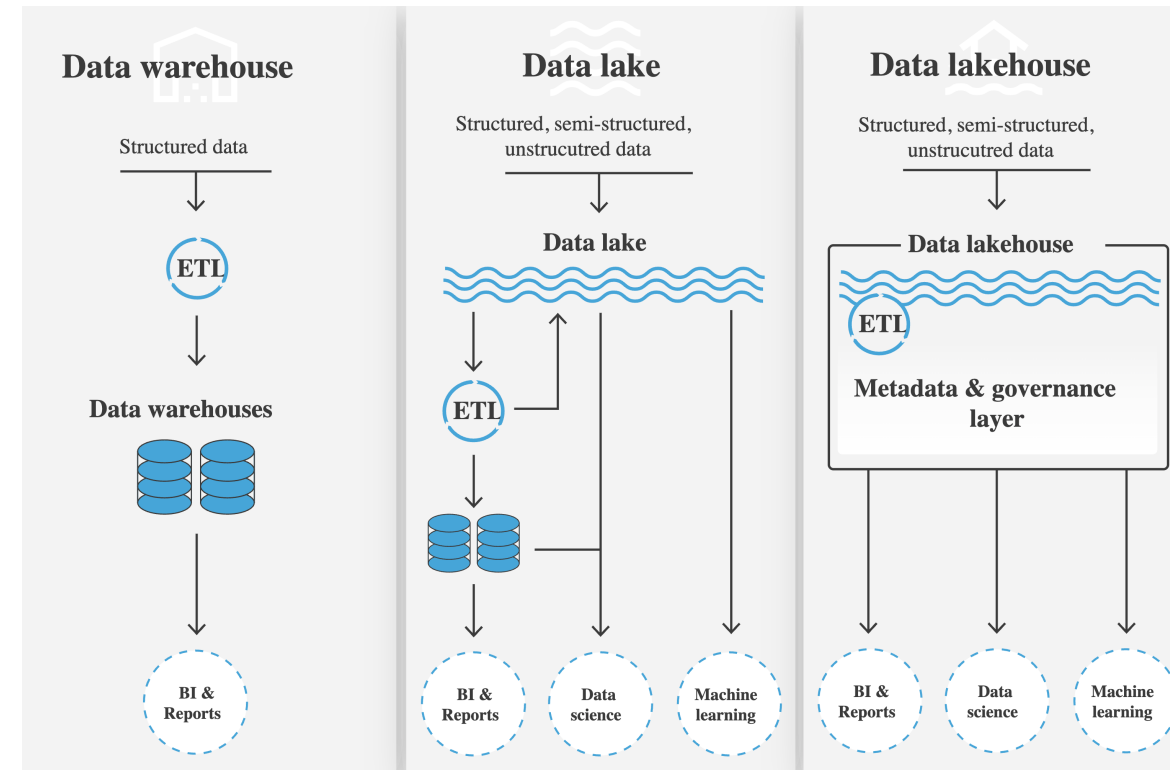
From Big Data stacks to data platforms

- First: ad-hoc scripts + single DB
- Then: “Big Data” ecosystems
- Now: data platforms / modern data stacks
 - Separation of compute/storage
 - Modular engines (batch, SQL, streaming)
 - Governance + observability built-in
- TEAD is also changing...



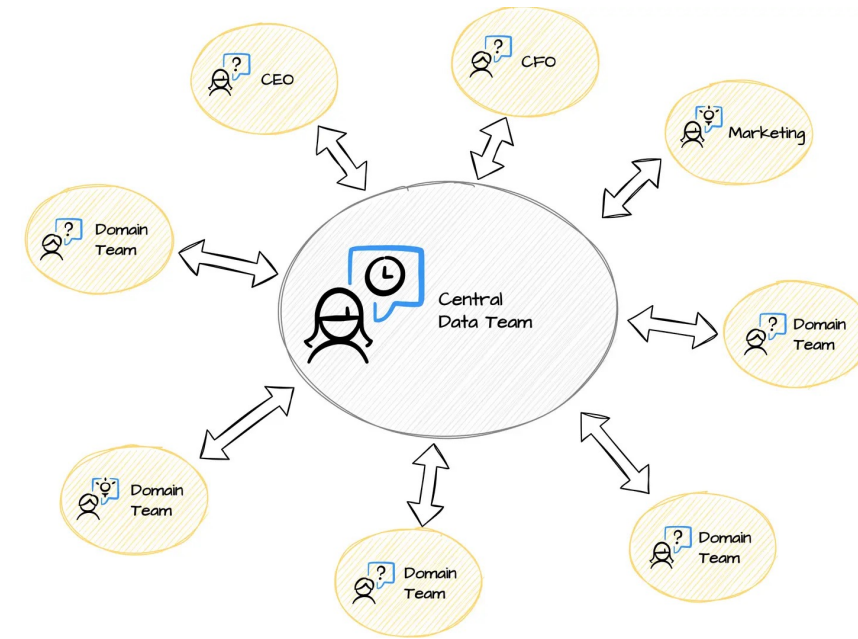
Reference architectures

- **Data Warehouse:** A highly structured repository optimized for fast SQL queries and business intelligence using cleaned, pre-processed data
- **Data Lake:** A low-cost, flexible storage space that holds massive amounts of raw data in its native format for data science and large-scale processing
- **Data Lakehouse:** A modern hybrid that applies the performance and governance of a warehouse directly onto the scalable, low-cost storage of a lake

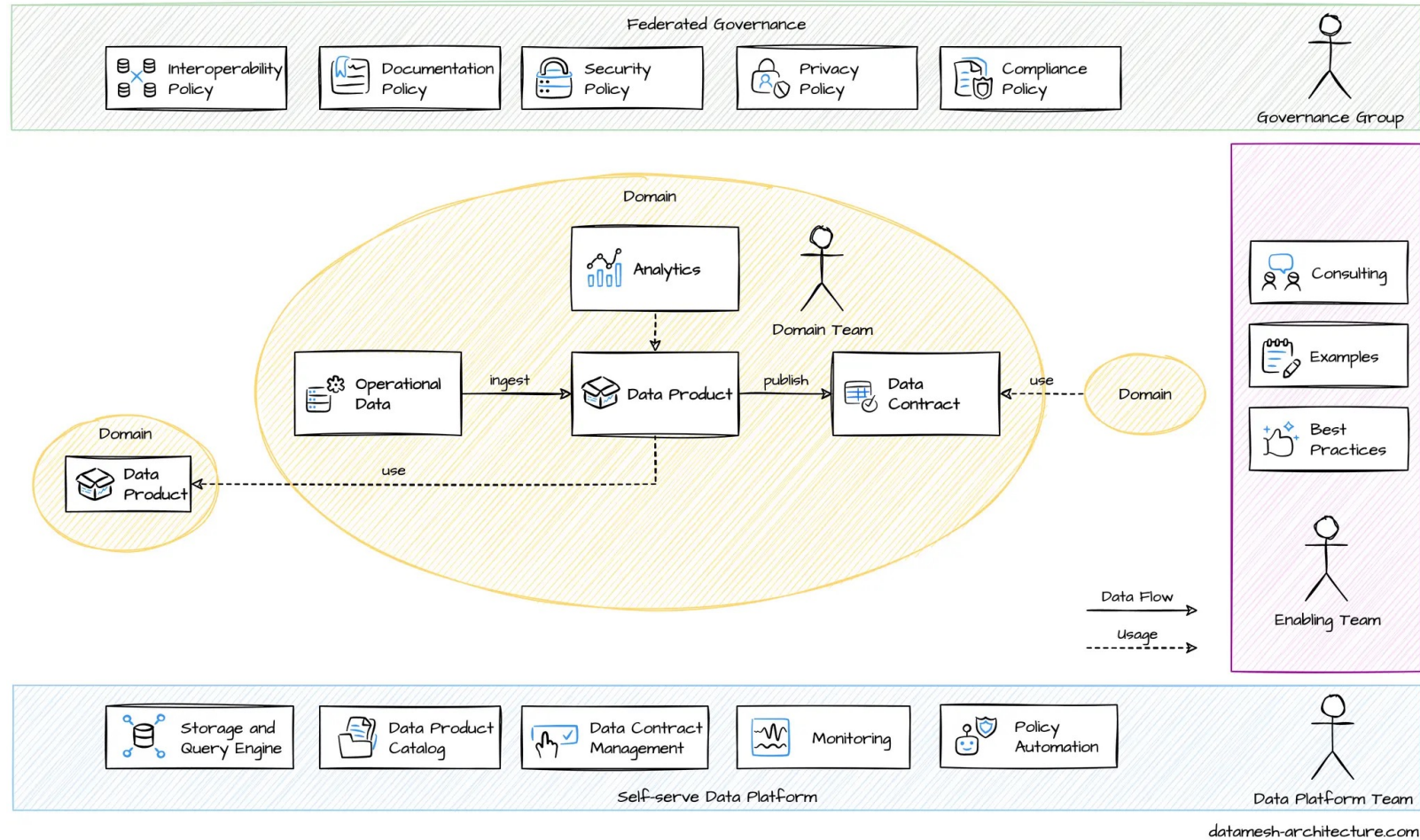


More recently... Data Mesh

- While a Data Lakehouse is a technology (how to store and query data), Data Mesh is a strategy (how to manage and own data across a large company)
 - **Domain-Oriented Ownership:** Instead of being centralized, each business unit (e.g., Sales, Marketing, HR) owns and manages its own data (although with a central infrastructure)
 - **Data as a Product:** Data is treated with the same care as a customer-facing product, meaning it must be discoverable, reliable, and easy to use for others in the company
 - **Self-Serve Data Infrastructure:** A central platform team provides the tools (like a Lakehouse) so that business units can manage their data without needing to be IT experts
 - **Federated Computational Governance:** Global standards (like security and naming conventions) are automated across all domains to ensure everything stays compatible.



Data Mesh Architecture



2. About us

Learning outcomes

- By the end of this course, you should be able to:
 - Identify when scalability + non-functional requirements change the architecture
 - Choose between lake/warehouse/lakehouse based on trade-offs
 - Design pipelines with contracts, versioning, and layered data models
 - Optimize processing (shuffle, skew, partitions, materialization choices)
 - Operate production-grade pipelines (idempotency, retries, backfills, observability)
 - Integrate data quality, governance, and (optionally) ML components

What you should be able to avoid

- After finishing this course you should be able to prevent common causes of “data pain”
 - Pipelines that break silently
 - Dashboards that disagree
 - “Hero engineers” + tribal knowledge
 - Costs that explode with no explanation
 - Changes that break downstream consumers

Industry roles you're training for

- Data Engineering
 - Focuses on building and maintaining the foundational infrastructure and automated pipelines that move raw data reliably and at scale from sources to storage
- Analytics Engineering
 - Operates at the intersection of data and business, transforming raw data into clean, documented, and reusable models within a semantic layer for consistent reporting
- Data Platform
 - Provides the centralized toolkit and ecosystem (including orchestration, governance, and cost monitoring) that enables data and analytics engineers to work efficiently and securely

How the course works

- Theory/practice sessions: concepts + patterns + architectural decision-making
 - Concepts + reference architectures
 - Case discussions (trade-offs: cost/latency/reliability/security)
 - Guided problem-solving micro-exercises
 - Short demos (illustrating concepts, not tool-centric)
- Labs: hands-on, project-based implementation
 - A longitudinal scenario
 - Implement an end-to-end scalable architecture (containerized)
 - Orchestrate production-grade pipelines
 - Validate with evidence: performance, quality, reliability, reproducibility
- Evaluation
 - 60% group project (9.5+)
 - 40% written test (at the same date of the exam) (7.5+)

The plan (high-level view)

1. Contract-Driven Design and Data Flow Modeling
2. Storage + table formats
3. Processing patterns + optimization
4. Orchestration + production reliability
5. Serving: SQL/APIs/semantic layer
6. Quality + observability + cost
7. Incremental + streaming
8. ML integration
- 9. Governance + evolution

1. Contract-Driven Design and Data Flow Modeling

- From the problem to analytical questions and metrics (data products)
- DAGs, determinism, and reproducibility in data pipelines
- Data contracts: schemas, keys, semantics, granularity, SLAs/SLOs
- Schema evolution and versioning; change management and compatibility
- Bronze/silver/gold layers and responsibilities per layer

2. Storage + table formats

- Object storage vs. filesystem vs. databases: operational and cost trade-offs
- Columnar formats (Parquet/ORC), compression, and pushdown
- Partitioning, clustering, and file management (small files, compaction)
- Tables and metadata in a lakehouse (Iceberg/Delta/Hudi, concepts and use cases)
- Catalog, metadata, and lineage (concepts and practical implications)

3. Processing patterns + optimization

- Transformation patterns: filters, aggregations, joins, windowing
- Shuffle, skew, parallelism, and mitigation strategies
- Materialization vs. query-on-read, caching and persistence management
- Measurement and evidence: profiling, execution plans, and job metrics
- Prototyping vs. production: limits, risks, and best practices

4. Orchestration + production reliability

- Orchestration vs. processing: responsibilities and limits
- Flyte concepts: tasks, workflows, launch plans, packaging, and execution
- Idempotency, retries, timeouts, backfills, and temporal parameterization
- Environment configuration, secrets, and reproducibility
- Versioning, caching, and rapid iteration, execution traceability

5. Serving: SQL/APIs/semantic layer

- Serving patterns: marts, OLAP vs. OLTP, distributed query engines
- SQL as an interface: metric governance and reuse
- Serving via query engines (e.g. Trino) vs. database materialization (e.g. Postgres)
- APIs and data products: contracts, versioning, and limits
- Visualization and reporting: design principles for analytical consumption

6. Quality + observability + cost

- Data quality: dimensions, per-layer checks, validation gates
- Pipeline testing: unit/integration/contract tests, synthetic data
- Observability: structured logs, metrics, tracing, and runbooks
- Incident management: data downtime, severity, and response
- Performance and cost: evidence-driven tuning, cost per pipeline/query

7. Incremental + streaming

- Incrementality: snapshots, upserts/merge, deduplication, logical watermarking
- CDC (Change Data Capture) and pipeline integration, handling late data
- Streaming: concepts (event time vs. processing time), delivery semantics
- Event-driven architectures (e.g. Kafka/Redpanda – overview)
- Criteria for choosing batch vs. incremental vs. streaming

8. ML integration

- Feature engineering and temporal validation, leakage and baselines
- Training reproducibility and pragmatic evaluation
- Batch scoring vs. online serving: trade-offs
- Tracking and artifact management (Minimum Viable MLOps)
- Integrating ML into the orchestrated pipeline (e.g. MLFlow)

9. Governance + evolution

- PII, compliance, and retention policies, auditing
- Access control and environment segregation
- Lineage and documentation as a data product component
- Change management: migrations, breaking changes, and communication
- Roadmap, technical debt, and platform maturity criteria

The stack



Technology	Role	Key Syllabus Modules
Minio	The physical "bucket" where all raw, transformed, and model data lives.	Scalability, Object Storage
Apache Iceberg	Adds a database-like layer to S3. Handles schema evolution, ACID transactions, and time travel	Contracts, Lakehouse, Incrementality
Trino	The high-speed SQL layer. It allows Analytics Engineers to query Iceberg tables across S3 without moving data	Query-on-read, Serving, Cost/Performance
Flyte	Manages the DAGs. It ensures pipelines are reproducible, idempotent, and can handle complex ML workflows	DAGs, Orchestration, ML Integration
MLflow	Tracks experiments, versions models, and manages the model registry	Feature Engineering, Tracking, MLOps

Scalable Technologies for Data Analysis

Introduction / Presentation

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026